APPENDIX

Table of Content

A Extended Application Examples

R1.O3, M.C2 Example 1 highlights an application, where using traditional CM sketches can cause unfairness in an urban traffic setting. Such examples are not limited to this setting, and can be found across a wide range of domains. In this section, we further highlight some of the other application examples across a diverse set of domains, where additive error guarantees are not satisfactory, motivating the need for (multiplicative) approximation factor and fair count min.

1) Wild Life Monitoring: Automatic monitoring of wildlife in conservation areas is essential for collecting statistics needed to protect and manage species across conservation categories (e.g., *endangered*, *threatened*, and *least concern*). Because manual monitoring is costly and difficult to scale, conservation efforts increasingly rely on automated data collection using resource-constrained sensors such as camera traps. These devices often have limited memory, energy, and communication capacity, making it impractical to store or transmit all detections.

To address this, each sensor can maintain approximate detection counts using a traditional Count-Min sketch, where element types correspond to animal species and each detection is an event. While the CM Sketch provides accurate estimates for frequent, least-concern species (e.g., reporting the 1528 detection of crow as 1607), its *additive error guarantees can severely inflate counts for rare endangered species* (e.g., increasing the 12 detection of Chetahs to 109), leading to misleading results. In contrast, an FCM offers multiplicative approximation guarantees, enabling accurate frequency estimation across all species groups, including endangered ones.

2) k-Shingle Frequency in a Document repository: A common task in large-scale document repositories is to estimate the frequency of k-shingles (a sequence of k consecutive words) across a massive collection of text documents. Since the number of possible shingles grows rapidly with vocabulary size and k, and storing exact counts is often infeasible, memory-efficient streaming summaries such as the Count-Min Sketch are widely used for approximate frequency estimation.

In practice, CM sketches provide reliable estimates for stop words, that are highly frequent. However, its additive error guarantees can significantly distort the estimated frequencies of technical terms or domain-specific terminology, making the results less useful for downstream tasks such as topic discovery, keyword extraction, or technical content analysis. This limitation motivates the use of Fair Count-Min (FCM), which offers multiplicative approximation guarantees and thus enables accurate estimation across both frequent and rare shingles.

3) Misleading Recommendations: In targeted advertising, recommendations are often driven by users' historical purchase behavior, such as the frequency of purchases within specific item categories (e.g., sports clothing). However, due to privacy constraints and limited storage, it may be infeasible to maintain complete transaction histories for all users. This motivates the use of compact streaming summaries, such as Count-Min Sketches, to approximately track purchase frequencies over time.

While CM sketches can efficiently estimate counts for frequent buyers, their additive error guarantees may substantially inflate frequency estimates for low-activity users. This can lead to misleading recommendations and incorrect targeting, resulting in wasted advertising budget and reduced campaign effectiveness.

4) Trend Detection in Social Media Streams: On platforms like Twitter, hashtags appear as a rapid, high-volume data stream. Automated or troll-bot hashtags can generate millions of occurrences per hour, while rare but potentially critical hashtags may surface only a few thousand times. In a CM, a small additive error can severely distort counts for the rarer tags. Although this may seem like a "free boost", the resulting collision-induced overcounts undermine reliability, making it hard to determine whether a rare hashtag is truly trending or merely inflated by overlap with high-volume tags. As a result, trend detection, resource allocation, and moderation decisions such as filtering become noisier—false signals may emerge while genuinely urgent topics are delayed or misclassified.

5) Threat Detection in Network Traffic Monitoring: In network monitoring, IP addresses can often be divided into two groups: benign and malicious. While benign IPs (e.g., widely used services such as Google DNS) may generate massive volumes of traffic, malicious IPs often appear only sporadically. In such settings, a Count-Min Sketch (CM) with a small additive error has little effect on estimating the frequencies of high-traffic benign IPs. However, the same additive error can significantly inflate the estimated counts of low-frequency malicious IPs due to hash collisions, masking meaningful anomalies. As a result, subtle but critical security threats may be harder to detect reliably.

6) Performance Measurement for Small Advertisers: In online advertising, platforms track how often each ad or user click event occurs. Popular ads may generate millions of impressions, while niche ads may only get a few hundred. With CM, small additive errors hardly affect big advertisers but can completely distort the performance metrics for small advertisers — for example, inflating a niche ad's 50 clicks into 500 due to collisions. This creates unfair reporting, potentially wasting budgets or hiding click fraud.

B Missing Proofs

Proof of Lemma 1. Let C_i be the total frequency counts in each low-frequency bucket $i \in [w_{\mathbf{g}_1}]$. That is, $C_i = \sum_{e_j:\mathrm{h}(e_j)=i} f(e_j)$.

For every element $e_j \in \mathbf{g}_1$, the value of the bucket it hashed to is returned as its frequency estimation. That is, $\hat{f}(e_j) = C_{\mathrm{h}(e_j)}$. Therefore, following the approximation factor definition, $\hat{f}(e_j) = \frac{f(e_j)}{\alpha(e_j)} = C_{\mathrm{h}(e_j)}$. Hence,

$$\alpha(e_j) = \frac{f(e_j)}{C_{\mathrm{h}(e_j)}} \tag{11}$$

Therefore,

$$\begin{aligned}
\mathbb{E}\left[\alpha_{g_1}\right] &= \frac{1}{n_{g_1}} \sum_{e_j \in g_1} \alpha(e_j) = \frac{1}{n_{g_1}} \sum_{e_j \in g_1} \frac{f(e_j)}{C_{h(e_j)}} \\
&= \frac{1}{n_{g_1}} \sum_{i=1}^{w_{g_1}} \sum_{e_j : h(e_j)=i} \frac{f(e_j)}{C_i} && \text{//breaking the calculation per cell} \\
&= \frac{1}{n_{g_1}} \sum_{i=1}^{w_{g_1}} \frac{1}{C_i} \sum_{e_j : h(e_j)=i} f(e_j) && \text{//factoring out the common term} \\
&= \frac{1}{n_{g_1}} \sum_{i=1}^{w_{g_1}} \frac{1}{C_i} C_i && \text{//replacing sum of frequencies with } C_i \\
&= \frac{1}{n_{g_1}} \sum_{i=1}^{w_{g_1}} 1 = \frac{w_{g_1}}{n_{g_1}} && (12)
\end{aligned}$$

Similarly, the expected approximation factor for the group $g_2$can be computed as

$$\mathbb{E}\left[\alpha_{g_2}\right] = \frac{w_{g_2}}{n_{g_2}} = \frac{w - w_{g_1}}{n - n_{g_1}}$$

□

Proof of Lemma 2. We aim to find the expected value of Y:

$$\mathbb{E}[Y] = \sum_{x=1}^{n} xP(Y = x)$$

Since the domain of the random variable Y is the nonnegative integers $\{1, 2, \cdots\}$, the expected value can alternatively be computed using the following equation [40].

$$\mathbb{E}[Y] = \sum_{x=1}^{n} P(Y \geq x)$$

$$\Pr(Y \geq x) = \Pr(\min(X_1, \cdots, X_d) \geq x) = \Pr(X_1 \geq x, \cdots, X_d \geq x).$$

Since $X_1, \cdots, X_d$ are independent and identically distributed, the probability is computed as:

$$\begin{aligned}
\Pr(Y \geq x) &= \Pr(X_1 \geq x, \cdots, X_d \geq x) \\
&= \prod_{i=1}^{d} \Pr(X_i \geq x) = \Pr(X \geq x)^d
\end{aligned}$$

where $\Pr(X = x) = \binom{n_g}{x} \cdot \left(\frac{1}{w_g}\right)^x \cdot \left(\frac{w_g - 1}{w_g}\right)^{n_g - x}$. Hence, the probability $\Pr(X \geq x)$ is:

$$\Pr(X \geq x) = \sum_{i=x}^{n_g} \binom{n_g}{i} \left(\frac{1}{w_g}\right)^i \left(\frac{w_g - 1}{w_g}\right)^{n_g - i}.$$

Putting everything together, we obtain the expected value of Y as:

$$\begin{aligned}
\mathbb{E}[Y] &= \sum_{x=1}^{n_g} \Pr(X \geq x)^d \\
&= \sum_{x=1}^{n_g} \left[\sum_{i=x}^{n_g} \binom{n_g}{i} \left(\frac{1}{w_g}\right)^i \left(\frac{w_g - 1}{w_g}\right)^{n_g - i} \right]^d . && (13)
\end{aligned}$$

□

Proof of Lemma 3. Define the random variable s_j that indicates if the j-th element in the observed stream belongs to group $\mathbf{g}$. That is, For each of the observed elements $e_{S_i}, i \in \{1, \ldots, m\}$, define the indicator random variable R2.Q9, M.C3

$$Y_i = \begin{cases} 1 & e_{S_i} \in \mathbf{g}, \\ 0 & \text{otherwise.} \end{cases}$$

The probability of drawing an element from the group $\mathbf{g}$ is $\Pr(Y_i = 1) = \frac{n_g}{n} = p$. Hence $Y_i \sim$ Bernoulli(p) and $\mathbb{E}[Y_i] = p$.

The total number of observed elements from group $\mathbf{g}$ is $x_g = \sum_{i=1}^{m} Y_i$. By linearity of expectation,

$$\mathbb{E}[x_g] = \sum_{i=1}^{m} \mathbb{E}[Y_i] = \sum_{i=1}^{m} p = mp.$$

Now consider the estimator $\widehat{n}_g = n \cdot \frac{x_g}{m}$. Taking expectation yields

$$\mathbb{E}[\widehat{n}_g] = \frac{n}{m}\mathbb{E}[x_g] = \frac{n}{m}(mp) = np = n \cdot \frac{n_g}{n} = n_g.$$

✓ Therefore, $\widehat{n}_g$ is an unbiased estimator of n_g.

Next, let us prove the tail bound. Define $\widehat{p} := x_g/m$. Note that $\widehat{n}_g = nx_g/m = n\widehat{p}$. Applying Hoeffding's inequality, we get

$$\Pr(\widehat{p} - p \geq \epsilon) \leq \exp(-2m\epsilon^2) \text{ and } \Pr(p - \widehat{p} \geq \epsilon) \leq \exp(-2m\epsilon^2).$$

By a union bound, $\Pr(|\widehat{p} - p| \geq \epsilon) \leq 2\exp(-2m\epsilon^2)$.

✓ As a result, since $\widehat{n}_g - n_g = n(\widehat{p} - p)$, hence

$$\Pr(|\widehat{n}_g - n_g| \geq n\epsilon) = \Pr(|\widehat{p} - p| \geq \epsilon) \leq 2\exp(-2m\epsilon^2).$$

□

Proof of Lemma 4. In order to analyze the runtime of our algorithm, we assume a modification of the real-RAM model of computation where every basic arithmetic operation is computed in $O(1)$ time. We note that for any pair of positive integer numbers q, k, it holds that $\binom{q}{k} = \prod_{i=1}^{k} \frac{q+1-i}{i}$ so $\binom{q}{k}$ can be computed in $O(k)$ time. Finally, the k-th power of any real number can be computed in $O(\log k)$ time.

We analyze the runtime in one iteration of the binary search. By definition of our model of computation, we compute b_1, t_1 in $O(1)$ time and s_1 in $O(\log n)$ time. Then α_1 is computed in $O(1)$ time. For every value of $i \in \{2, \ldots, n_{g_1}\}$, given $b_{i-1}, t_{i-1}, s_{i-1}$ the values of b_i, t_i, s_{i-1} and hence α_i are computed in $O(1)$ time. So all values α_i for $i \in \{1, \ldots, n_{g_1}\}$ are computed in $O(n + \log n) = O(n)$ time in total. Next, we observe that given β_{i+1}, the value β_i is computed in $O(1)$ time, so all values β_i are computed in $O(n)$ time. Finally, the sum $\sum_{x=1}^{n_{g_1}} \beta_x^d$ is computed in $O\left(\sum_{x=1}^{n} \log(d)\right) = O(n \log d)$ time. The binary search is executed for $O(\log n)$ iterations, so in total the running time of our algorithm is $O(n \log(n) \log(d))$. Notice that usually, $n \gg d$ so the total running time is bounded by $O(n \log^2 n)$.

□

R2.O2

Proof of Proposition 1. Let $p = \frac{1}{w} \in (0, 1]$. Let $X_p \sim$ Bin(n, p), where Bin(n, p) denotes the binomial distribution with n trials and success probability p. For any integer $i \in \{0, 1, \ldots, n\}$, we have $\Pr[X_p = i] = \binom{n}{i} p^i (1-p)^{n-i}$. For each integer $j \in \{1, \ldots, n\}$, define $h_j(p) = \sum_{i=j}^{n} \binom{n}{i} p^i (1-p)^{n-i} = \Pr[X_p \geq j]$. With this notation, we can rewrite $\mathcal{E}(n, d, w)$ as $\mathcal{E}(n, d, w) = \sum_{j=1}^{n} \left(h_j\left(\frac{1}{w}\right)\right)^d$.

For every j, the function $h_j(p) = \Pr[X_p \geq j]$ is increasing in p because as the success probability p increases, the probability of observing at least j successes in a binomial experiment also increases. Since $p = \frac{1}{w}$ is decreasing in w, it follows that $h_j\left(\frac{1}{w}\right)$ is decreasing in w. Consequently, $\left(h_j\left(\frac{1}{w}\right)\right)^d$ is decreasing in w. Because $\mathcal{E}(n, d, w)$ is a sum of functions that are each decreasing in w, we conclude that $\mathcal{E}(n, d, w)$ is decreasing with respect to w. □

Proof of Lemma 5. Let us begin by computing $\mathcal{L}_{CM}$, the total additive error for the standard CM sketch. By Chapter 3 of [29], the expected additive error of an element type $e_j \in \mathcal{U}$ is

$$\varepsilon_A(e_j) = \frac{N - f(e_j)}{w}.$$

Then, the total additive error of the standard CM sketch is computed as

$$\mathcal{L}_{CM} = \sum_{j=1}^{n} \varepsilon_A(e_j) = \sum_{j=1}^{n} \frac{N - f(e_j)}{w} = n\frac{N}{w} - \frac{N}{w} = (n-1)\frac{N}{w} \quad (14)$$

Now, let us compute $\mathcal{L}_{FCM}$, the total additive error for the FCM sketch.

$$\mathcal{L}_{FCM} = \sum_{j=1}^{n} \varepsilon_A(e_j) = \sum_{e_j \in \mathrm{g}_1} \varepsilon_A(e_j) + \cdots + \sum_{e_j \in \mathrm{g}_\ell} \varepsilon_A(e_j)$$

Following the same calculation as in Equation (14), for every group $\mathrm{g} \in \mathcal{G}$,

$$\sum_{e_j \in \mathrm{g}} \varepsilon_A(e_j) = (n_\mathrm{g} - 1)\frac{N_\mathrm{g}}{w_\mathrm{g}}$$

Hence,

$$\mathcal{L}_{FCM} = \sum_{e_j \in \mathrm{g}_1} \varepsilon_A(e_j) + \cdots + \sum_{e_j \in \mathrm{g}_\ell} \varepsilon_A(e_j) = \sum_{\mathrm{g} \in \mathcal{G}} (n_\mathrm{g} - 1)\frac{N_\mathrm{g}}{w_\mathrm{g}}$$

From Theorem 1, we know, $w_\mathrm{g} = \frac{n_\mathrm{g}}{n} w, \forall \mathrm{g} \in \mathcal{G}$, while $\sum_{\mathrm{g} \in \mathcal{G}} N_\mathrm{g} = N$. Therefore,

$$\begin{aligned}
\mathcal{L}_{FCM} &= \sum_{\mathrm{g} \in \mathcal{G}} (n_\mathrm{g} - 1)\frac{N_\mathrm{g}}{w_\mathrm{g}} \\
&= \sum_{\mathrm{g} \in \mathcal{G}} (n_\mathrm{g} - 1)\frac{N_\mathrm{g}}{\frac{n_\mathrm{g}}{n} w} = \sum_{\mathrm{g} \in \mathcal{G}} n\left(1 - \frac{1}{n_\mathrm{g}}\right)\frac{N_\mathrm{g}}{w} \\
&= n\sum_{\mathrm{g} \in \mathcal{G}} \frac{N_\mathrm{g}}{w} - \sum_{\mathrm{g} \in \mathcal{G}} \frac{n}{n_\mathrm{g}} \cdot \frac{N_\mathrm{g}}{w} = n\frac{N}{w} - \sum_{\mathrm{g} \in \mathcal{G}} \frac{n}{n_\mathrm{g}} \cdot \frac{N_\mathrm{g}}{w} \quad (15)
\end{aligned}$$

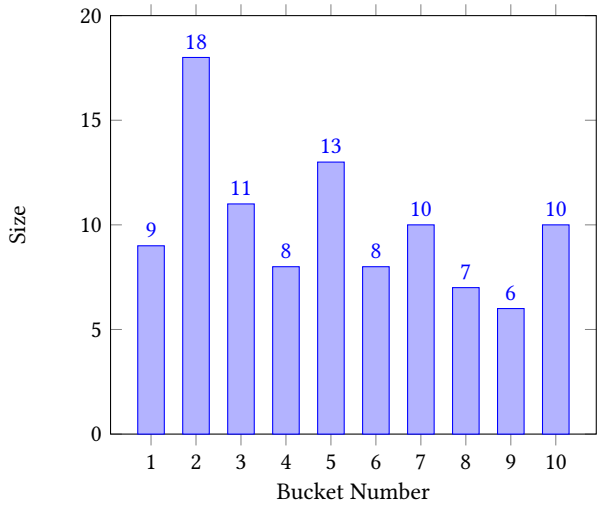

Figure 41: Illustration of a random distribution of $n = 100$ elements to $w = 10$ buckets.

Interestingly, combining Equations 9, 14, and 15, we show that $PoF < 0$, for a CM with random hashing scheme:

$$\begin{aligned}
PoF &= \mathcal{L}_{FCM} - \mathcal{L}_{CM} \\
&= n\frac{N}{w} - \sum_{\mathrm{g} \in \mathcal{G}} \left(\frac{n}{n_\mathrm{g}} \cdot \frac{N_\mathrm{g}}{w}\right) - (n-1)\frac{N}{w} \\
&= \frac{N}{w} - \sum_{\mathrm{g} \in \mathcal{G}} \frac{n}{n_\mathrm{g}} \cdot \frac{N_\mathrm{g}}{w} \\
&< \frac{N}{w} - \sum_{\mathrm{g} \in \mathcal{G}} \frac{N_\mathrm{g}}{w} \qquad \text{//since } \frac{n}{n_\mathrm{g}} > 1, \forall \mathrm{g} \in \mathcal{G} \\
&= \frac{N}{w} - \frac{N}{w} = 0
\end{aligned}$$

□

C Price of Fairness: Randomness vs. Uniformity

In Section 5, we observed a counterintuitive result for $d = 1$, proving a negative price of fairness (PoF) for FCM, compared to a regular CM sketch that uses a random hash function. This result can be explained by the fact that a random hash function is unlikely to *uniformly* distribute the elements to the buckets (an interesting related topic is *the occupant problem* [63]). As a result, some of the buckets will have more than $\frac{n}{w}$ elements in them. For example, Figure 41 illustrates a random hashing of $n = 100$ element types into $w = 10$ buckets. While a uniform distribution would allocate 10 elements to each bucket, the random hashing allocated up to 18 elements in one bucket.

Elements in large buckets will suffer from a large additive error, and there are a large number of them in such buckets. Hence, such buckets significantly increase the total additive error of the CM.

FCM increases the size-uniformity of the buckets by reserving $w_\mathrm{g} = \frac{n_\mathrm{g}}{n} w$ for each group g. For example, Figure 42 illustrates the allocation of the 100 elements to the 10 buckets, by dividing (arbitrarily) the elements into two groups of size $n_{\mathrm{g}_1} = 50$, $n_{\mathrm{g}_2} = 50$.

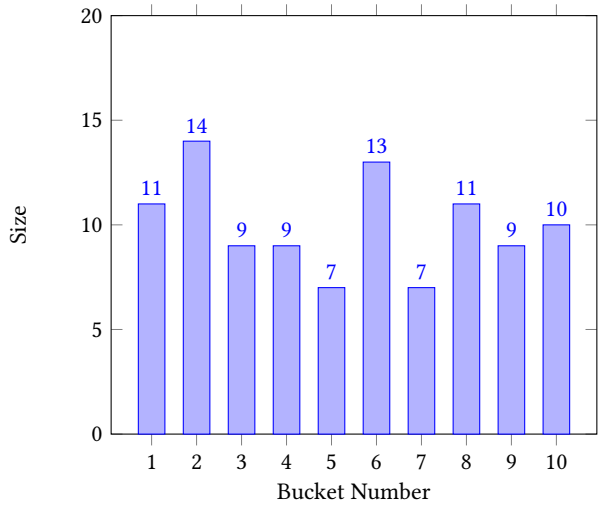

Figure 42: Illustration of a random distribution of $n = 100$ elements with two groups of sizes $n_{g_1} = 50$ and $n_{g_2} = 50$ to $w = 10$ buckets, based on FCM.

As a result, each group is allocated 5 buckets, and the distribution of the elements is more uniform, reducing the maximum size of the buckets (in this example) to 14. Increasing the uniformity (reducing the change of large-size buckets) helps to reduce the total additive error of FCM versus CM.

C.1 PoF for Uniform hashing ($d = 1$)

Using a hashing scheme (such as data-informed hashmaps [54]) that ensures uniformity by allocating exactly $\frac{n}{w}$ elements to each bucket can resolve the non-uniformity issue in the CM sketches. In the following, we compute the PoF of FCM for $d = 1$ when a uniform hashing scheme is used.

Since the hashing is uniform, the size of each bucket i in the CM sketch is

$$C_i = \frac{n}{w}$$

Therefore, each element e_j collides with exactly $\frac{n}{w} - 1$ other elements. In other words, the frequency of each element contributes to the additive error of exactly $\frac{n}{w} - 1$ other elements.

As a result, the total additive error can be computed as

$$\begin{aligned}\mathcal{L}_{CM} &= \sum_{j=1}^{n} \varepsilon_A(e_j) \\ &= \sum_{j=1}^{n} f(e_j)\left(\frac{n}{w} - 1\right) = \left(\frac{n}{w} - 1\right)\sum_{j=1}^{n} f(e_j) \\ &= N\left(\frac{n}{w} - 1\right) \end{aligned} \tag{16}$$

Now, let us compute $\mathcal{L}_{FCM}$, the total additive error for the regular fair-count-min sketch.

$$\mathcal{L}_{FCM} = \sum_{j=1}^{n} \varepsilon_A(e_j) = \sum_{e_j \in \mathrm{g}_1} \varepsilon_A(e_j) + \cdots + \sum_{e_j \in \mathrm{g}_\ell} \varepsilon_A(e_j)$$

Following the same calculation as in Equation 16, for every group $\mathrm{g} \in \mathcal{G}$,

$$\sum_{e_j \in \mathrm{g}} \varepsilon_A(e_j) = N_{\mathrm{g}}\left(\frac{n_{\mathrm{g}}}{w_{\mathrm{g}}} - 1\right)$$

Hence,

$$\mathcal{L}_{FCM} = \sum_{e_j \in \mathrm{g}_1} \varepsilon_A(e_j) + \cdots + \sum_{e_j \in \mathrm{g}_\ell} \varepsilon_A(e_j) = \sum_{\mathrm{g} \in \mathcal{G}} N_{\mathrm{g}}\left(\frac{n_{\mathrm{g}}}{w_{\mathrm{g}}} - 1\right)$$

From Theorem 1, we know, $w_{\mathrm{g}} = \frac{n_{\mathrm{g}}}{n} w, \forall \mathrm{g} \in \mathcal{G}$, while $\sum_{\mathrm{g} \in \mathcal{G}} N_{\mathrm{g}} = N$. Therefore,

$$\begin{aligned}\mathcal{L}_{FCM} &= \sum_{\mathrm{g} \in \mathcal{G}} N_{\mathrm{g}}\left(\frac{n_{\mathrm{g}}}{w_{\mathrm{g}}} - 1\right) \\ &= \sum_{\mathrm{g} \in \mathcal{G}} N_{\mathrm{g}}\left(\frac{n_{\mathrm{g}}}{w_{\mathrm{g}}}\right) - \sum_{\mathrm{g} \in \mathcal{G}} N_{\mathrm{g}} = \sum_{\mathrm{g} \in \mathcal{G}} N_{\mathrm{g}}\left(\frac{n_{\mathrm{g}}}{w_{\mathrm{g}}}\right) - N \\ &= \left(\frac{n}{w}\right)\sum_{\mathrm{g} \in \mathcal{G}} N_{\mathrm{g}} - N \qquad \text{//since } \frac{n_{\mathrm{g}}}{w_{\mathrm{g}}} = \frac{n}{w}, \forall \mathrm{g} \in \mathcal{G} \\ &= N\left(\frac{n}{w}\right) - N = N\left(\frac{n}{w} - 1\right) \end{aligned} \tag{17}$$

Finally, combining Equations 9, 16, and 17, we get,

$$PoF = \mathcal{L}_{FCM} - \mathcal{L}_{CM} = N\left(\frac{n}{w} - 1\right) - N\left(\frac{n}{w} - 1\right) = 0 \tag{18}$$

That is, the PoF is zero for $d = 1$, when a uniform hashing scheme is used.

C.2 PoF $d = 1$ vs. $d > 1$

Table 2 provides our experiment results on synthetic datasets with two groups and different group sizes, providing a comparison of CM and FCM with respect to additive errors and the price of fairness across different n_{g_1} values for $d = 1$ and $d = 5$.

D Dataset Descriptions

We use five sufficiently-large benchmark datasets for our experiments:

NYC Bus [83]: The NYC Bus Dataset originates from the MTA real-time data stream, capturing detailed information about buses across NYC. Collected at approximately 10-minute intervals, each record includes a bus's location, route, stop, and other operational details, along with its scheduled arrival time from the MTA timetable. This dataset contains approximately 27 million records, from which we sample 10% to form our stream. We construct the stream by combining the bus line attribute, `PublishedLineName`, with the bus ID attribute, `VehicleRef`, resulting in 53,622 distinct element types. Each element type belongs to one of two groups corresponding to the MTA's bus subsidiaries: *NYCT* or *MTABC*.

DDoS [69]: This dataset was compiled from several public IDS datasets, including CSE-CIC-IDS2018-AWS, CICIDS2017, and the CIC DoS 2016 dataset. It contains approximately 13 million records, each labeled as either a DDoS request or benign. To create the stream, we sample 20% of the records and use the `Src IP` attribute as the element type, resulting in 23,211 distinct element types. Each element belongs to one of two groups: *Benign* or *DDoS*.

Total Additive Error			n_{g_1} (n = 10,000)								
			9000	**8000**	**7000**	**6000**	**5000**	**4000**	**3000**	**2000**	**1000**
$d = 1$	theoretical	**CM**	19,047,019	28,054,816	37,081,001	46,092,289	55,121,050	64,147,002	73,186,882	82,323,243	91,265,426
		FCM	18,985,224	28,028,987	37,003,399	46,050,674	55,115,538	64,089,586	73,165,549	82,249,909	91,228,281
	empirical	**CM**	19,039,343	28,085,806	37,053,427	46,096,489	55,113,194	64,234,176	73,227,052	82,283,374	91,328,074
		FCM	18,991,509	27,982,573	37,002,246	46,071,394	55,086,890	64,133,815	73,168,189	82,230,649	91,276,271
		PoF	↓-47,834	↓-103,233	↓-51,181	↓-25,095	↓-26,304	↓-100,361	↓-58,863	↓-52,725	↓-51,803
$d = 5$	empirical	**CM**	7,964,348	11,572,548	16,458,026	22,023,181	28,305,699	34,442,308	40,959,703	47,230,290	54,257,770
		FCM	11,695,556	17,114,933	22,666,115	28,053,739	33,856,154	39,448,942	44,913,211	50,089,698	55,893,637
		PoF	↑3,731,208	↑5,542,385	↑6,208,089	↑6,030,558	↑5,550,455	↑5,006,634	↑3,953,508	↑2,859,408	↑1,635,867

Table 2: comparison of CM and FCM w.r.t additive errors and the price of fairness across different n_{g_1} values for $d = 1$ and $d = 5$.

Reddit-Twitter [53]: This dataset contains approximately 12 million text records, each associated with a source. We preprocess the dataset by retaining only records from either *Reddit* or *Twitter*. The text is cleaned by removing stopwords and lemmatizing tokens, with each token associated with its source. We also deduplicate the data, ensuring that each word is linked to a single source using majority selection. This process results in a stream of 35 million words, each belonging to one of two groups: *Reddit* or *Twitter*.

Census [59]: This dataset contains a one percent sample of the *Public Use Microdata Samples* person records derived from the complete 1990 census dataset. It includes around 2.5 million records with 68 categorical attributes. We use this dataset for experiments involving arbitrarily defined groups. Entity types are defined by concatenating the values of `iYearsch`, `dIndustry`, and the grouping attribute. For binary group experiments, the `iSex` attribute is used to define the groups resulting in 430 element types. To extend the grouping to ℓ groups, we utilize other attributes with higher cardinalities.

Google N-Grams [61]: This dataset is a large-scale collection of n-gram frequency counts extracted from texts in books digitized through the Google Books project. It has been used in related research to evaluate frequency estimation sketches. In our evaluations, we use a sample of 2-grams that begin with $a\_$ to generate a stream of approximately 1.25 million element types, with a total frequency of around 5 million. We utilize this dataset for experiments in which groups are defined based on the frequency of elements associated with a given element type. If an element's frequency is below a specified threshold, it is assigned to group $\mathbf{g}_1$; otherwise, it is assigned to group $\mathbf{g}_2$. This approach is further extended to partition elements into ℓ frequency-based groups.

E Additional Experiment Results

In this section, we present comprehensive experimental results across five datasets: 1) GOOGLE NGRAM, 2) CENSUS, 3) NYC BUS, 4) DDOS, 5)REDDIT-TWITTER. We also provide the absolute values of the approximation factors and additive errors for each group to facilitate a clearer understanding of the trends in unfairness and the associated price of fairness under each setting.

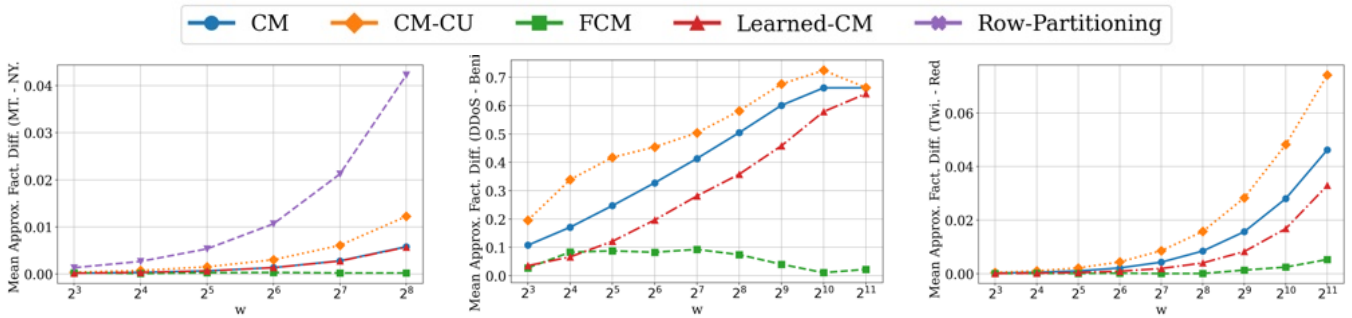

Figure 43: effect of varying w on unfairness, NYC BUS, $d = 5$.

Figure 44: effect of varying w on unfairness, DDOS, $d = 5$.

Figure 45: effect of varying w on unfairness, REDDIT-TWITTER, $d = 5$.

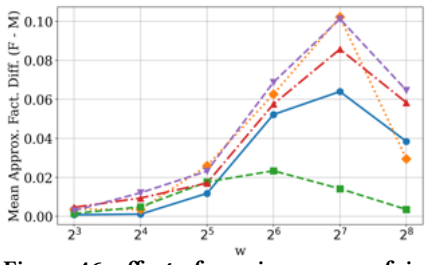

Figure 46: effect of varying w on unfairness, CENSUS, $d = 5$.

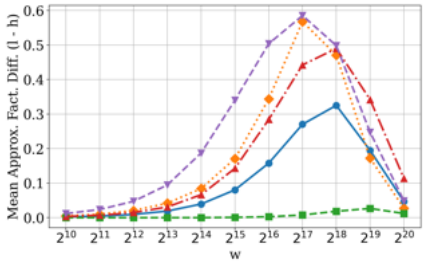

Figure 47: effect of varying w on unfairness, GOOGLE NGRAM, $d = 5$.

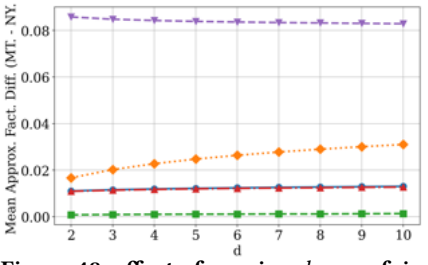

Figure 48: effect of varying d on unfairness, NYC BUS, $w = 512$.

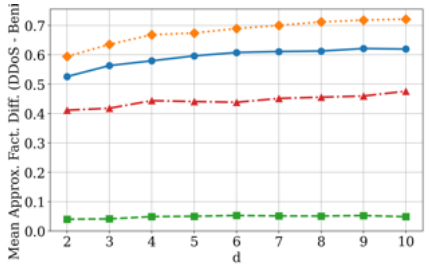

Figure 49: effect of varying d on unfairness, DDOS, $w = 64$.

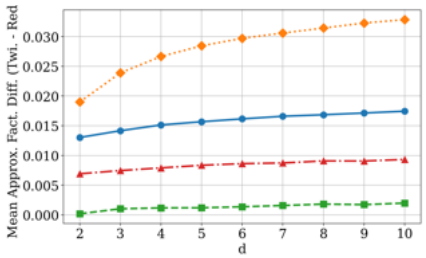

Figure 50: effect of varying d on unfairness, REDDIT-TWITTER, $w = 512$.

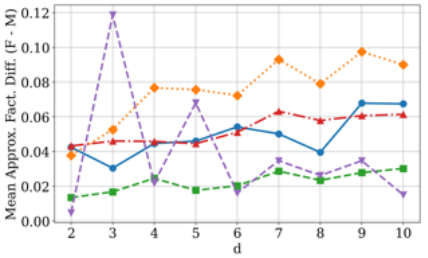

Figure 51: effect of varying d on unfairness, CENSUS, $w = 64$.

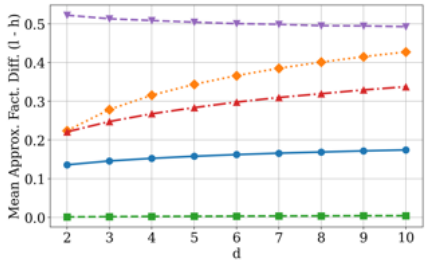

Figure 52: effect of varying d on unfairness, GOOGLE NGRAM, $w = 65536$.

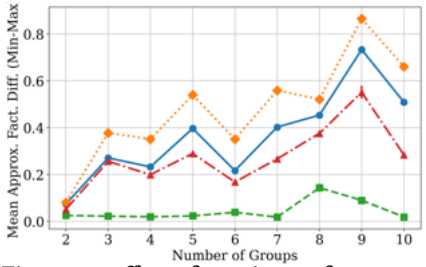

Figure 53: effect of varying # of groups ℓ on unfairness, CENSUS, $w = 64, d = 5$.

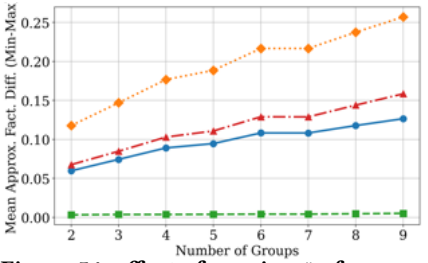

Figure 54: effect of varying # of groups ℓ on unfairness, GOOGLE NGRAM, $w = 65536, d = 5$.

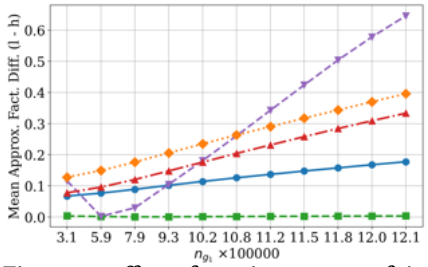

Figure 55: effect of varying n_{g_1} on unfairness, GOOGLE NGRAM, $w = 65536, d = 5$.

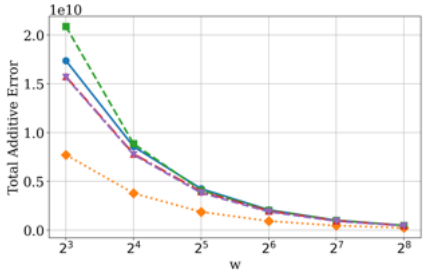

Figure 56: effect of varying w on price of fairness, NYC BUS, $d = 5$.

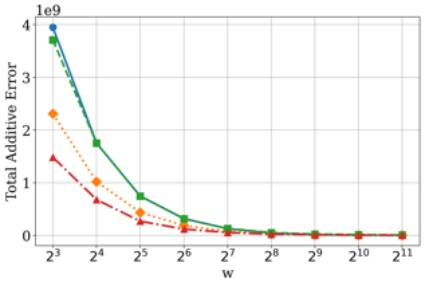

Figure 57: effect of varying w on price of fairness, DDOS, $d = 5$.

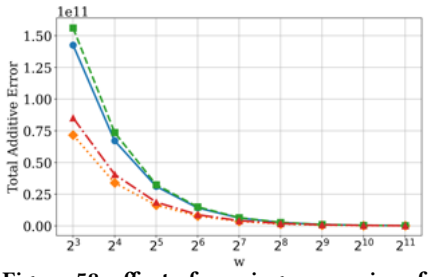

Figure 58: effect of varying w on price of fairness, REDDIT-TWITTER, $d = 5$.

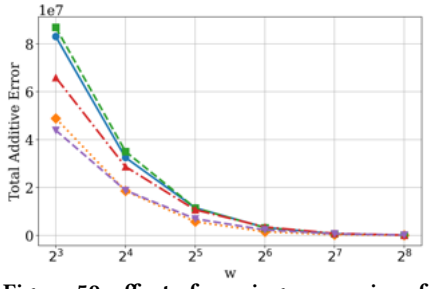

Figure 59: effect of varying w on price of fairness, CENSUS, $d = 5$.

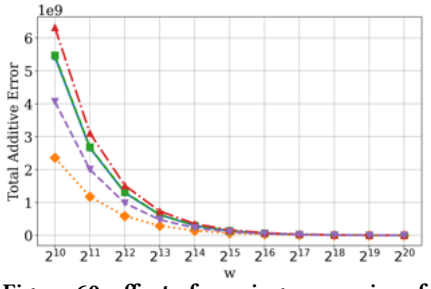

Figure 60: effect of varying w on price of fairness, GOOGLE NGRAM, $d = 5$.

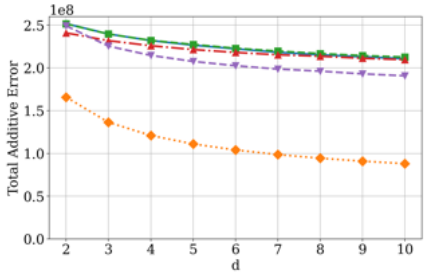

Figure 61: effect of varying d on price of fairness, NYC BUS, $w = 512$.

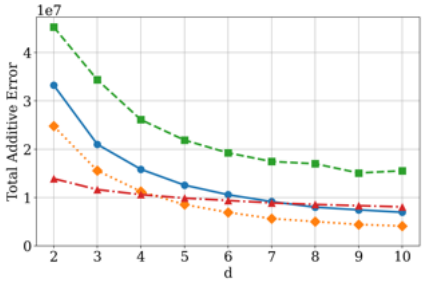

Figure 62: effect of varying d on price of fairness, DDOS, $w = 64$.

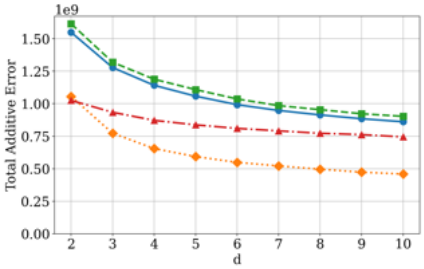

Figure 63: effect of varying d on price of fairness, REDDIT-TWITTER, $w = 512$.

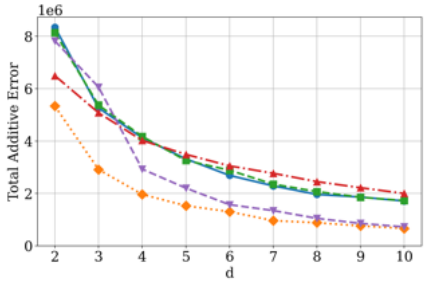

Figure 64: effect of varying d on price of fairness, CENSUS, $w = 64$.

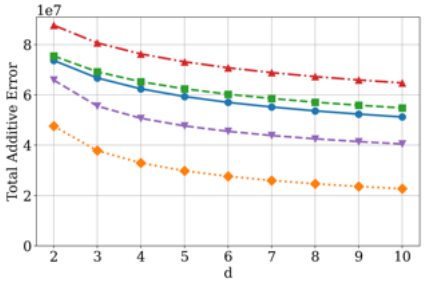

Figure 65: effect of varying d on price of fairness, GOOGLE NGRAM, $w = 65536$.

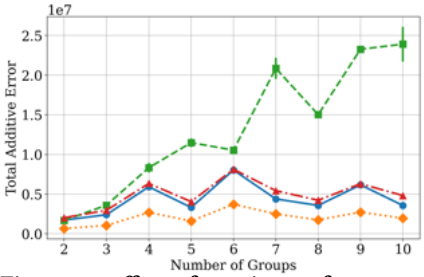

Figure 66: effect of varying # of groups ℓ on price of fairness, CENSUS, $w = 64$, $d = 5$.

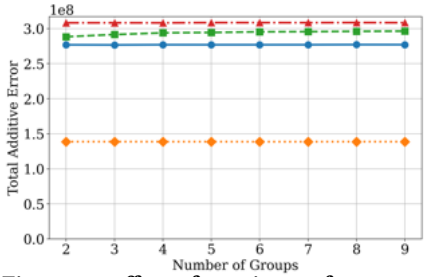

Figure 67: effect of varying # of groups ℓ on price of fairness, GOOGLE NGRAM, $w = 65536$, $d = 5$.

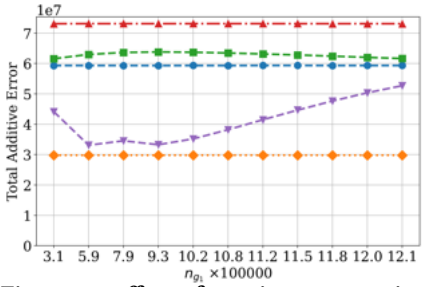

Figure 68: effect of varying n_{g_1} on price of fairness, GOOGLE NGRAM, $w = 65536$, $d = 5$.

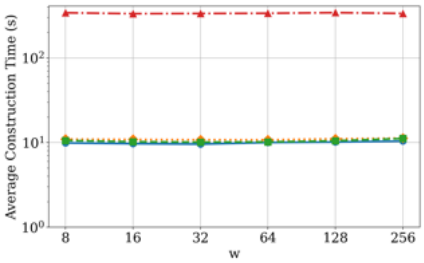

Figure 69: effect of varying w on construction time, NYC BUS, $d = 5$.

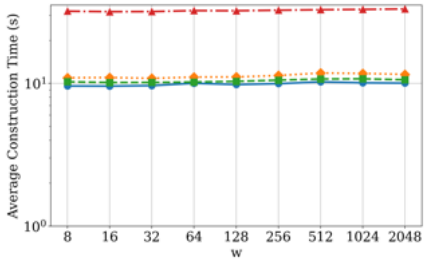

Figure 70: effect of varying w on construction time, DDOS, $d = 5$.

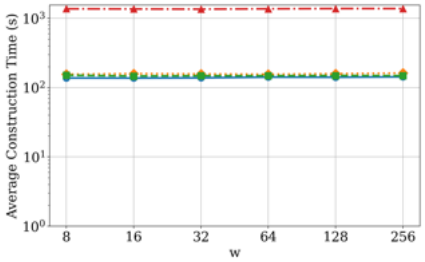

Figure 71: effect of varying w on construction time, REDDIT-TWITTER, $d = 5$.

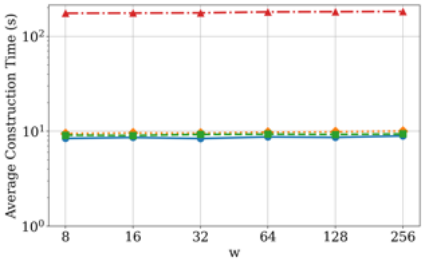

Figure 72: effect of varying w on construction time, CENSUS, $d = 5$.

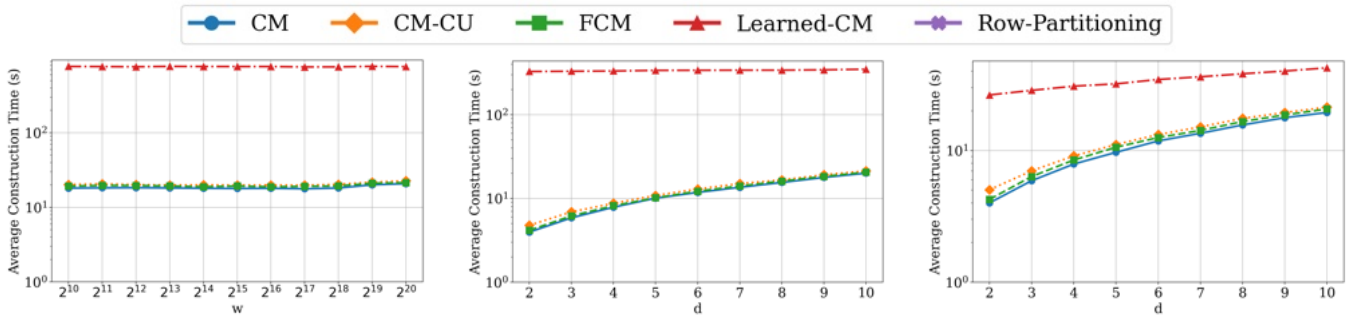

Figure 73: effect of varying w on construction time, GOOGLE NGRAM, $d = 5$.

Figure 74: effect of varying d on construction time, NYC BUS, $w = 512$.

Figure 75: effect of varying d on construction time, DDOS, $w = 64$.

Figure 76: effect of varying d on construction time, REDDIT-TWITTER, $w = 512$.

Figure 77: effect of varying d on construction time, CENSUS, $w = 64$.

Figure 78: effect of varying d on construction time, GOOGLE NGRAM, $w = 65536$.

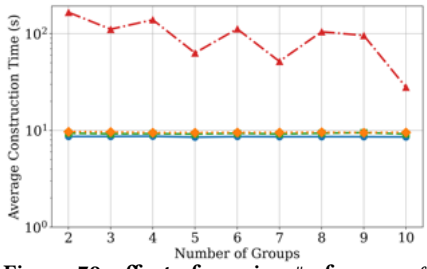

Figure 79: effect of varying # of groups ℓ on construction time, CENSUS, $w = 64, d = 5$.

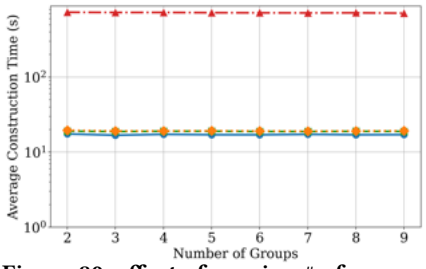

Figure 80: effect of varying # of groups ℓ on construction time, GOOGLE NGRAM, $w = 65536, d = 5$.

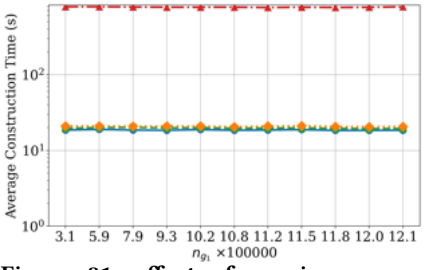

Figure 81: effect of varying n_{g_1} on construction time, GOOGLE NGRAM, $w = 65536, d = 5$.

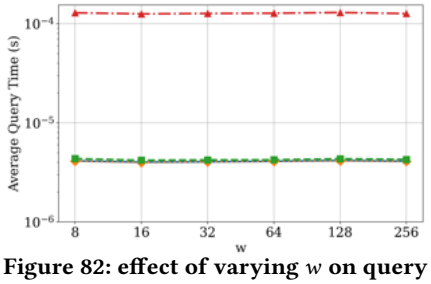

Figure 82: effect of varying w on query time, NYC BUS, $d = 5$.

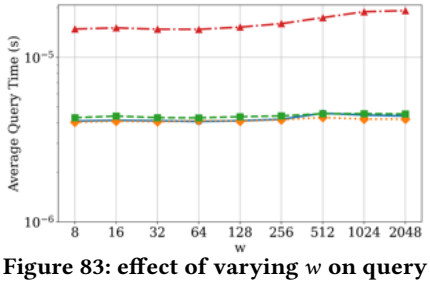

Figure 83: effect of varying w on query time, DDOS, $d = 5$.

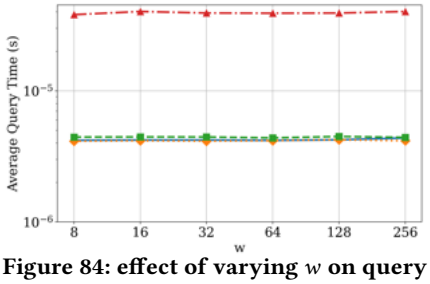

Figure 84: effect of varying w on query time, REDDIT-TWITTER, $d = 5$.

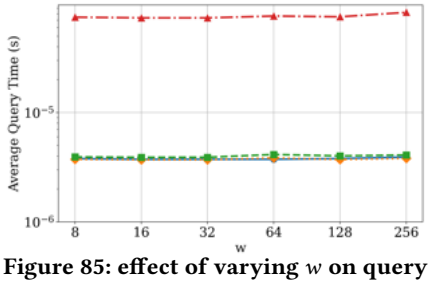

Figure 85: effect of varying w on query time, CENSUS, $d = 5$.

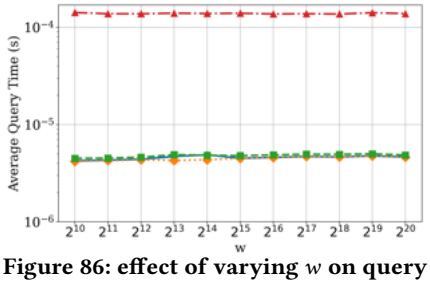

Figure 86: effect of varying w on query time, GOOGLE NGRAM, $d = 5$.

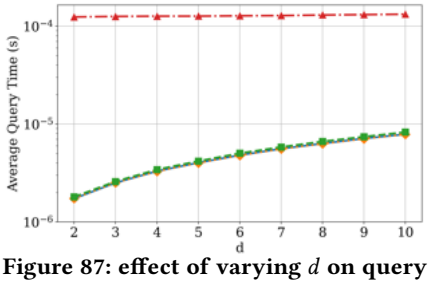

Figure 87: effect of varying d on query time, NYC BUS, $w = 512$.

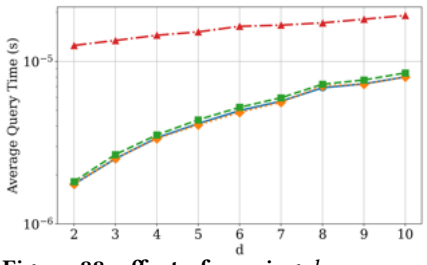

Figure 88: effect of varying d on query time, DDOS, $w = 64$.

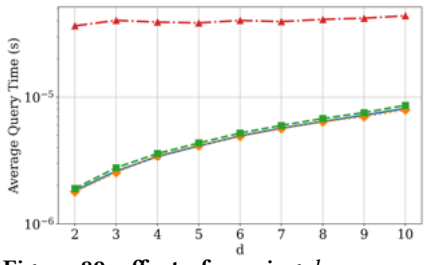

Figure 89: effect of varying d on query time, REDDIT-TWITTER, $w = 512$.

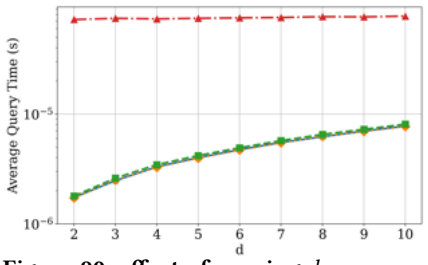

Figure 90: effect of varying d on query time, CENSUS, $w = 64$.

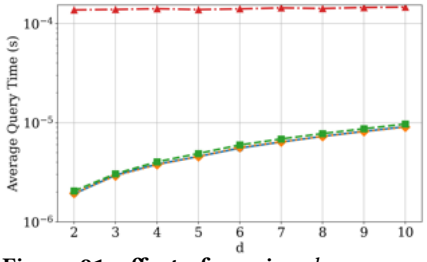

Figure 91: effect of varying d on query time, GOOGLE NGRAM, $w = 65536$.

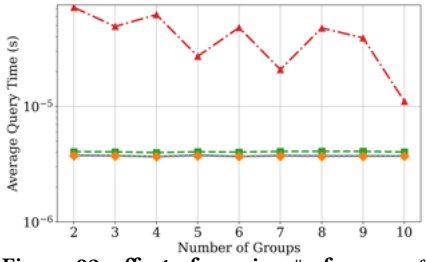

Figure 92: effect of varying # of groups ℓ on query time, CENSUS, $w = 64, d = 5$.

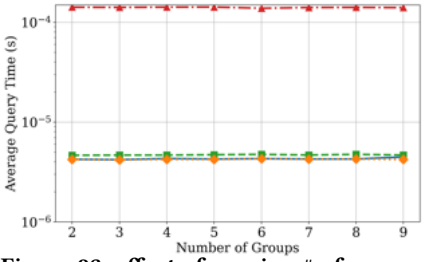

Figure 93: effect of varying # of groups ℓ on query time, GOOGLE NGRAM, $w = 65536, d = 5$.

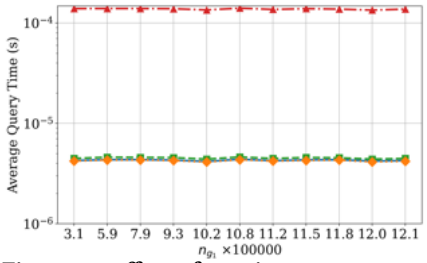

Figure 94: effect of varying n_{g_1} on query time, GOOGLE NGRAM, $w = 65536, d = 5$.

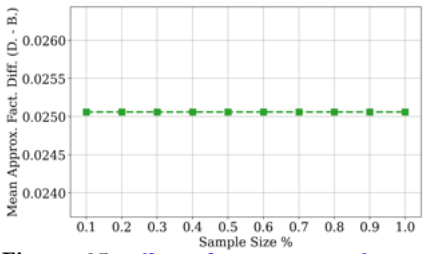

Figure 95: effect of varying sample size for estimating n_{g_1} on unfairness, DDOS, $w = 64, d = 1$.

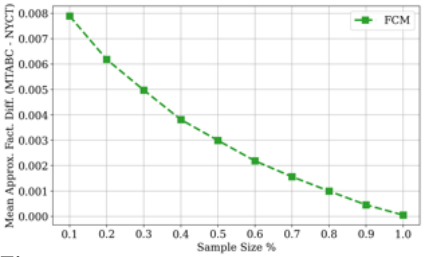

Figure 96: effect of varying sample size for estimating n_{g_1} on unfairness, NYC BUS, $w = 512, d = 1$.

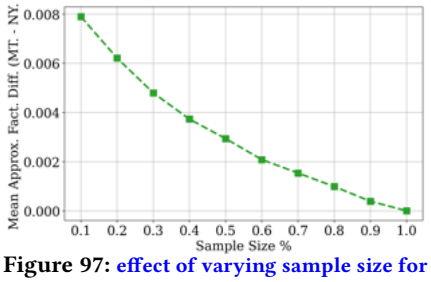

Figure 97: effect of varying sample size for estimating n_{g_1} on unfairness, NYC BUS, $w = 512, d = 5$.

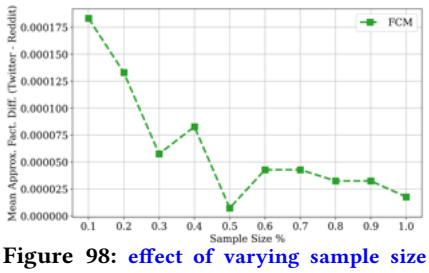

Figure 98: effect of varying sample size for estimating n_{g_1} on unfairness, REDDIT-TWITTER, $w = 512, d = 1$.

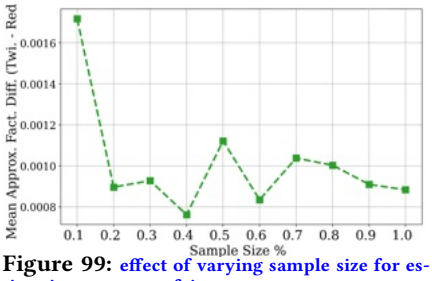

Figure 99: effect of varying sample size for estimating n_{g_1} on unfairness, REDDIT-TWITTER, $w = 512, d = 5$.

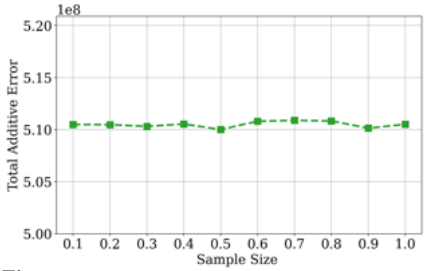

Figure 100: effect of varying sample size for estimating n_{g_1} on price of fairness, DDOS, $w = 64, d = 1$.

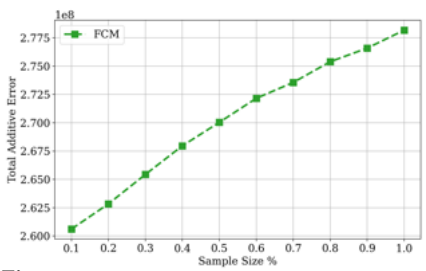

Figure 101: effect of varying sample size for estimating n_{g_1} on price of fairness, NYC BUS, $w = 512, d = 1$.

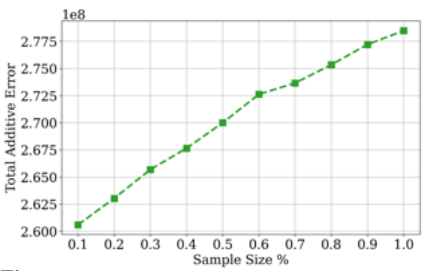

Figure 102: effect of varying sample size for estimating n_{g_1} on price of fairness, NYC BUS, $w = 512, d = 5$.

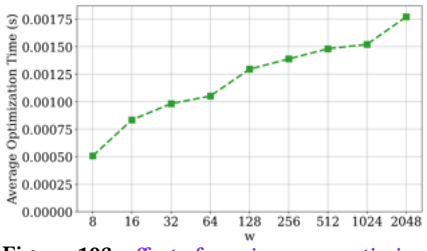

Figure 103: effect of varying w on optimization time, DDOS, $d = 5$.

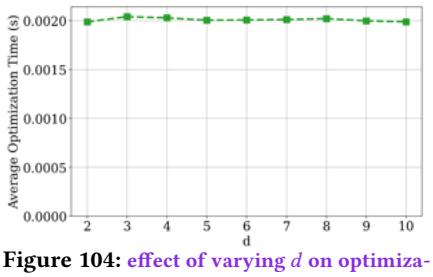

Figure 104: effect of varying d on optimization time, CENSUS, $w = 64$.

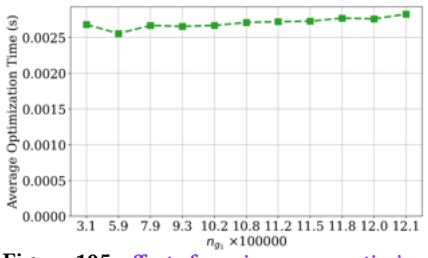

Figure 105: effect of varying n_{g_1} on optimization time, GOOGLE NGRAM, $w = 65536, d = 5$.

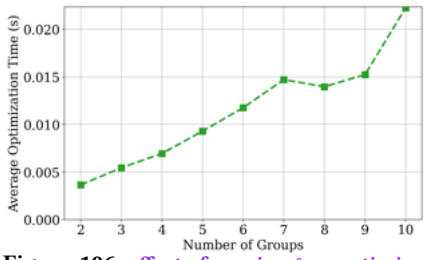

Figure 106: effect of varying ℓ on optimization time, CENSUS, $w = 64, d = 5$.